

Final Report for Blockchain Project

Jingyi Lv, Wenjie Huang, Xiang Gao

January 12, 2025

1 Introduction

In this project, we write a project to implement the basic blockchain structure and add some features to improve it. The blockchain is a decentralized and distributed ledger that records transactions across many computers. We implement the blockchain in GO language. We add a few example of malicious nodes to the blockchain to show its security. We also give an example of blockchain fork to show that the longest chain rule is correctly applied. And we test its efficiency in terms of the time it takes to add a new block to the chain under different difficulty levels (difficulty can be changed in the client).

In addition to the basic structure, we add three new features to the blockchain. First, we add fees to the transactions. The fees are paid to the miners who add the transactions to the blockchain to make miners more active. Second, we give the client the ability to add new nodes in the network. The client can also create connection between nodes. So theoretically, we can get any topology of nodes' connection we want. Finally, we add a graphical interface for the client so we do not need to type the inputs in the terminal and we can get some prompts about what we can do.

2 Design for Part 1

This section describes the basic structure of the blockchain.

We have two kinds of agents in the system: the client and the miner. The client is the user who wants to interact with the blockchain. The miner is the computer that stores the blockchain, mines new blocks and verifies the transactions.

In part 1, the miners are fully connected. Each miner has a mining thread. The miner thread needs to generate the nonce randomly. And when the hash function has enough power (0s at the beginning), (at `final_project21/src/miner/miner.go: 339`) the miner can create a block. And the block will be broadcast to all the miners.

When a miner receives a block (function `onNewBlockAppearance()`), it will verify the block. First it will check the block's power (stated before). Then it will check the block's transactions. If the block is valid, the miner will add the block to its blockchain (into `block_pool`). If the block is invalid, the miner will discard the block (not discarding the valid transactions in it).

The client can check the balance of every miner by inputting "query". The client can also add transactions to the blockchain. The format is to input "tx {source} {destination} {value}". Then the source will send the value to the destination in a transaction. The source will get the request of the transaction from the client. Then it will collect the

UTXOs of itself to reach the required value. Then it will put these UTXOs into the inputs of the transaction and sign each input with its private key. Then it will create an output with the public key of the destination. It will also create an output to itself if there is remained collected value. And it will sign all the outputs with its private key. (at final_project21/src/miner/miner.go: 548 581) Finally, it will broadcast the transaction to all the miners.

When the miners receive the transaction (function onNewTransactionAppearance()), they will put the transaction into the transaction_pool. And When a miner creates a new block, it will put the transactions in the transaction_pool into the block. It will also create a special transaction for itself, which is the coin creation reward. (at final_project21/src/miner/miner.go: 356,357) When validating the block, the miner will check the transactions in the block. (Just as what we stated before.) The validating method include whether the UTXOs are valid, whether the signature is correct and not stale, whether the input value is enough to cover the output value, etc. The coin creation reward must only appear once, and its value should be fixed. If the transaction is valid, the miner will add the transaction to the UTXO pool. If the transaction is invalid, the miner will discard the transaction.

To show the things more clearly, we let the miners print out every block with different formats when it's valid or invalid.

Another thing is about the longest chain rule. When a miner receives a block, it will check whether the block is in the longest chain. The checking method is to check whether the TotalDifficulty of the block is bigger than the current block at the main_chain_head. If it is, the miner will put it on the main_chain_head. And the whole main chain will be updated.

The Difficulty of each block is 2 to the power of the number of 0s at the beginning of the hash. The TotalDifficulty of each block is the sum of the Difficulty of all the blocks before it in a chain. The client can change the difficulty by inputting "adjust {difficulty}". Then the required number of 0s at the beginning of the hash will be the "difficulty" that has been input. The default difficulty is 24.

3 Settings for Experiments of Malicious or Lying or Forking Nodes

To prove that our blockchain can work securely and fairly, we need to test it with some bad nodes.

3.1 Malicious Nodes

We add three kinds of malicious nodes to the blockchain.

The first kind of malicious node (at final_project21/src/miner_malicious1/miner_malicious1.go) tries to pocket all the outputs of the transactions in the block it mines. It will change the destination of the outputs to itself. However, it cannot sign the outputs with the private key of the source. When the other miners receive the block, they will check the transactions in the block. They will find that the signature of the outputs are not valid. (Outputing "Output signature verification fail") So the transaction is invalid. The miners will discard the block.

The second kind of malicious node (at `final_project21/src/miner_mailcious2/miner_malicious2.go`) tries to double spend the UTXOs. When they are requested to create a transaction, they will create two transactions with the same UTXOs in the inputs. One transaction for the destination and one for itself. When the other miners receive the block containing the transactions, they will check the transactions in the block. They will find that the UTXOs are already spent. (Outputing “UTXO verification fail”) So the transaction is invalid. The miners will discard the block.

The third kind of malicious node (at `final_project21/src/miner_mailcious3/miner_malicious3.go`) tries to create transactions with stale signatures. They have a thread to create transactions with stale signatures, always using the same output to itself. The transactions will be broadcast to all the miners. When the other miners receive the block containing the transaction, they will check the transactions in the block. They will find that the signature is stale. (Outputing “Stale message”) So the transaction is invalid. The miners will discard the block.

Other malicious nodes can be added in the same way. Including nodes that tries to broadcast transactions with stale inputs (which will be directly rejected at `onNewTransactionAppearance()`), nodes that tries to broadcast blocks with invalid coin creation, etc. None of them threatens the security of the blockchain.

3.2 Lying Nodes

Sometimes the miners are not trying to have fake transactions, but to get more coin creation rewards, they may lie about their mining speed by creating blocks that do not have enough power. We create a miner that will create blocks with less power than the required difficulty but still claim the original difficulty. (at `final_project21/src/miner_lazy/miner_lazy.go`) When the other miners receive the block, they will check the block’s power. If the block does not have enough power, the miners will discard the block while outputting “PoW invalid”.

3.3 Forking Nodes

To show that the longest chain rule is correctly applied, we create a miner that will create a fork of the blockchain. (at `final_project21/src/miner_fork/miner_fork.go`), it should be used with two other miners. (at `final_project21/src/miner_honest/miner_honest.go`) The honest node can only mine at a low speed (mine with higher difficulty than requested at `miner_honest.go:342`). The fork node can mine at a high speed. However, the fork node start to broadcast the blocks it mined after 35 seconds. So the honest node can mine a few blocks before the fork node starts to broadcast. When the fork node starts to broadcast, the honest node will receive the blocks, which creates a longer chain. The honest node will put the blocks into the main chain. While the fork node can receive the blocks from the honest node from the beginning, it does not put these blocks on the main chain since their `TotalDifficulty` is smaller than its own blocks.

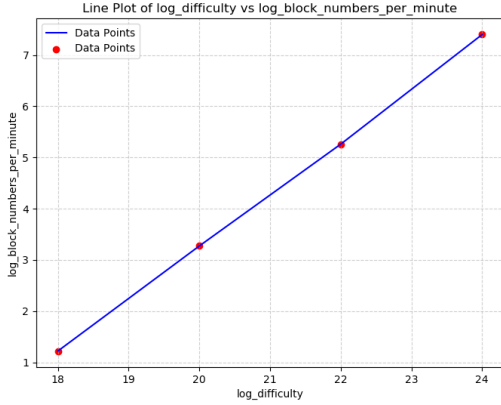
4 Prediction on Features can be Added in Part 2

In the next part, we will add some features to the blockchain to make it more efficient and convenient. We can add a feature to give priority to some of the transactions, so that the miners can choose which transactions to pack first (assuming that the block

can be full). We can also add a feature to allow nodes to work when they are not fully connected. The nodes can have different topologies, just like in real world. We can also try to give more prompts when the client is using the blockchain, so that the client can know what they can do and what they cannot do.

5 Results for the Efficiency of Blockchain

We test the blocks can be created in a minute under different difficulties. We repeat the test and get the average blocksper minute.



The result shows that the blockchain has a reasonable speed of creating blocks. The speed is faster when the difficulty is lower. And when the log of difficulty reduces by 2, the speed of creating blocks is about four times faster, which matches our prediction. This shows that the definition of difficulty in the system can show the workload of mining a block. This is very important, since we use the total difficulty to judge the longest chain.

Another discovery is that the speed of creating blocks is not strictly linear to the difficulty. When the difficulty is higher, the speed of creating blocks is slower than the linear prediction. A possible explanation is that the hash function has its randomness, and when the difficulty is higher, the number of blocks generated in a minute is smaller, so the randomness has a bigger impact on the result, resulting in a huger error.

6 Feature 1: Fee for Transaction

In the original design, the miners just pack all the transactions in the transaction_pool when they create a new block. However, when there are too many transactions in the transaction_pool that exceeds the capacity of the pool, the miners need to choose which transactions to pack, and it may set a sequence of priority. As a way to encourage the miners, we add a fee to each transaction. The fee is paid to the miner who mines the block containing the transaction by the “source”. The miner will pack the transactions with highest fees first until the transaction_pool is empty or the block is full.

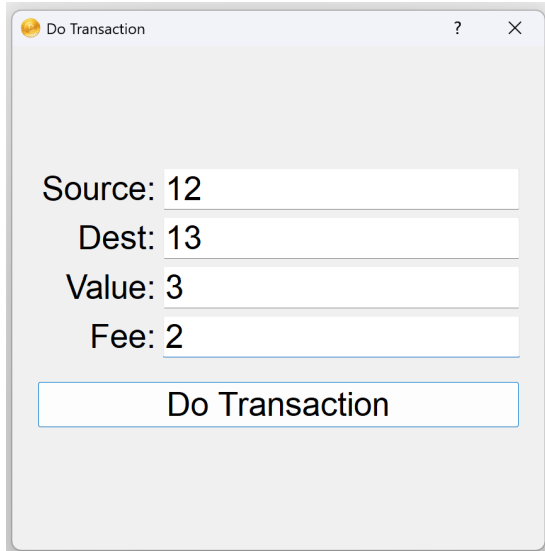


Figure 1: The graphical interface of transaction creation

Under this setting, we have some new assumptions:

- The client should input “tx {source} {destination} {value} {fee}” to create a transaction. (In the graphical interface, this means filling 4 blanks instead of 3, like Figure 1.)
- The fee should be non-negative.
- The sum of output values and the fee in a transaction cannot exceed the input values, not larger than the sum of values in the collected UTXOs.
- The miner should produce just one fee transaction in the block it mines for every transaction with a fee.
- The fee transaction cannot have a fee, the same for a coin creation.
- There is only one output in the fee transaction, and the value in it is just the fee of the transaction it corresponds to.

The Figure 2 is an example of how the fee works: We can see that the first block has the transactions with higher fees. The second block has the transactions with lower fees.



Figure 2: Two blocks with different fees

7 Feature 2: Add New Nodes

In the original design, the miners are fully connected. However, in the real world, the nodes can have different kinds of topologies. We add a feature that the client can add new nodes to the network. The client can also create connections between nodes.

When there is a new node, the client should get an input “newnode {miner_id}” (or just “Add New Miner” in the graphic interface, like in the Figure 3). The client will call an RPC service “Greet” to the miner to ensure that the miner is working (at final_project22/src/client/client.go:74). If the miner is working, the client will add the miner to the connection_pool.

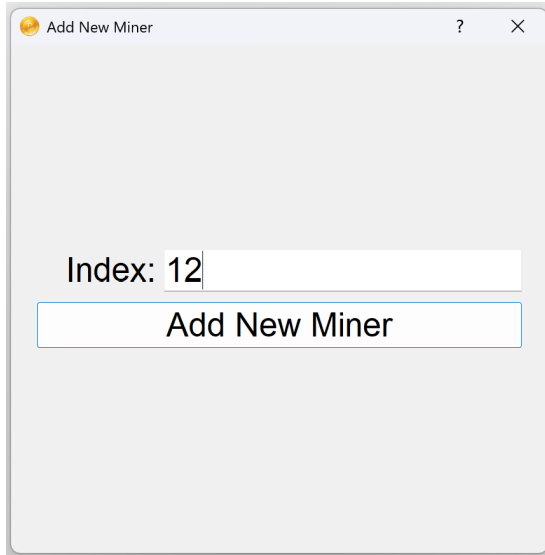


Figure 3: The graphical interface of adding new miner

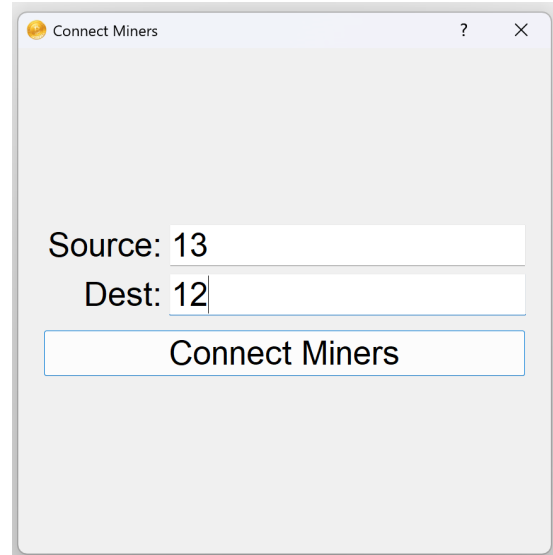


Figure 4: The graphical interface of creating connection

In the new setting, a new assumption is that a miner would not start to work when it is in a network of only one miner. This assumption is used to avoid too much forks when there are many single nodes. So a node should be added to the network by the client. When the client wants to create a connection between two nodes, the client should input “connect {source_id} {dest_id}”. Another assumption is that when the “connect” operation connects a new node into the network, the new node should be regarded as the source. When the “source” miner receives the RPC “ConnectTo” request, it can send a “ConnectFrom” request to the “dest” miner. The “dest” miner can judge whether the “source” is a new node (a bool variable “is_new”) based on the node_list it has. If “is_new” is not true, the two miners will simply be connected. Otherwise, the “dest” miner will send the “source” miner the whole blockchain it has (including the difficulty). The synchronization process including “SyncHead”, “NewSyncBlock” and “SyncTail”. When the synchronization is completed. A zero will be pushed into sync_signal channel to indicate the completion of the synchronization. The “source” miner will then start its own work.

8 Feature 3: GUI

In the original design, the users have to interact with the console to do transaction or query balance. However, it is not convenient to users, especially those who know little about programming. So, we implement a GUI (Graphical User Interface) to make users more comfortable, provide more prompts to available operations, as well as increase type-safety.

A design idea of this GUI is to make the interface Simple. So we do not squeeze all the operation buttons into the main menu, but lead the user step-by-step to what he/she want, and do not show surplus buttons in the middle.

Functionality.

The users of the GUI are divided into users and administrators.

Users can query balance and do transaction through the command buttons.

And, if some one enters some arguments with wrong type, it will detect and remind, whether in the frontend or in the backend whose error message is passed to the frontend through the network. For example, in the transaction, entering negative indices belong to the former one, while entering indices which are not yet connected to the client belong to the latter one.

Finally, the user and the administrator can set the public ip, client port, and username as used in ssh command. Do not forget to set correct client port before doing any operations. And, the administrator can flush temp (what it indeed means is told in the Implementation part below).

Implementation.

The GUI feature contains two parts: frontend and backend. We met some trade-offs in developing both parts, and finally choose to: optimize developing efficiency, balancing implementation elegance, and considering speed. This is because: DDL is urgent, elegant code is more readable, but speed does not matter too much because GUI is used by human who are much slower than CPUs.

The frontend is finally implemented in Python on PyQt5 and packed by pyinstaller into an exe file in Windows. The other choice, VB, was ruled out because for the shell command Python can run but VB cannot run.

The backend, well, is a simple communication system based on files. We use this to overcome the main technology barrier: client is always on, but frontend needs to call client at any time. Other choices, including using some advanced packages that provide more direct manipulation on running processes, are ruled out because: on one hand. they requires reading documents and possible building environments which has high uncertainty or high variance of developing time, on the other hand, here we do not need to implement an entire file-based communication system with universal functionalities.

Frontend.

The main window is `class BitcoinGUI(QWidget)`. Then, except for “Query Miner’s Balance” and “Flush Temp”, each operation button triggers a dialog that further asks for the arguments. The corresponding triggering functions are named by “to...”. The dialogs and corresponding triggering functions are:

- `class SettingsDialog(QDialog)    –    def toSettings(self)`
- `class TxDialog(QDialog)        –    def toTx(self)`
- `class AdjustDialog(QDialog)    –    def toAdjust(self)`
- `class ConnectDialog(QDialog)   –    def toConnect(self)`
- `class NewnodeDialog(QDialog)   –    def toNewnode(self)`

When the user enters arguments and clicks the button in the newly appeared dialog, the actual action starts. The functions doing actual actions are named by “do...”, including that of “Query Miner’s Balance” and “Flush Temp”. The functions are:

- `def doSet(self)`

- `def doFlushTemp(self)`
- `def doQuery(self)`
- `def doTx(self)`
- `def doAdjust(self)`
- `def doConnect(self)`
- `def doNewnode(self)`

Each operation shows its result in a message box `QMessageBox.information`, except Query which shows its result in a dialog class `QueryResultDialog(QDialog)` through a table `QTableWidget()` to display the query result more conveniently.

Backend.

In the deployed repository `final_project22_deployed`, there are 4 important files (from now on omit the path when referring to them):

- `build/client`: always on, monitoring `command.txt`, writes to `result.txt`
- `build/helper`: started by the frontend, writes to `command.txt`, monitoring `result.txt`
- `temp/command.txt`: contains timestamp, command, ending symbol
- `temp/result.txt`: contains timestamp, result, ending symbol

Note: Here we manually implement a Lamport-style integer-based logic timestamp to be more elegant and avoid physical timestamp conflict. Here the ending symbol, as well as the empty symbol when first deployed or flushed, are designed to avoid file asynchronization error, e.g. the helper wants to write “1 tx 2 3 456 78”, but when it writes to “1 tx 2 3 456 7” the client reads.

A whole lifecycle of a request is:

- user chooses the operation, enters arguments, and clicked the actual action button
- frontend do type-safe check and send `ssh` command to start remote `helper`
- `helper` write the command into `command.txt` and monitors `result.txt`
- `client`, monitoring `command.txt`, executes the command and outputs to `result.txt`
- `helper`, monitoring `result.txt`, returns the result
- frontend obtains the result and displays it on the GUI

9 Conclusion

In conclusion, we implemented a blockchain with security, efficiency and convenience. It has a reasonable speed of creating blocks and dealing with transactions. It is also quite robust facing different situations. And the new features, especially the graphical interface, make it more user-friendly. In the future, we will continue to improve the algorithm and resolve vulnerabilities if there are any.